

FACULTY OF ENGINEERING

Wireless Pendant

REII 327

By:

Mr EJ De Klerk

Student Number: 41942442

Date: 02 December 2025

Submitted in pursuit of the degree:

Bachelor of Engineering

In

Computer and Electronic Engineering

At

North-West University Potchefstroom Campus

Contents

List of Figures	3
List of Tables.....	4
1. Introduction	5
1.1 Purpose of the Wireless Controller	5
1.2 Role of the Wireless Controller in the Larger System.....	5
1.3 Scope of This Report	5
2. Design process	6
2.1 Context analysis	6
System context diagram (SCD)	6
SCD Explanation	6
2.2 System Architecture.....	8
Physical architecture diagram	8
Functional units.....	9
Interfaces	11
2.3 Functional and Behavioral Analysis	13
Mission description.....	13
FFBD Level 0.....	14
FFBD Level 1	15
2.4 Resource mapping.....	18
Validation discussion (unused/unsupported functions).....	18
2.5 Key Requirements.....	19
3. Implementation aspects.....	24
3.1 Trade study.....	24
Alternatives	24
Comparison matrix	25
Final decision justification.....	27
3.2 Calculations	28
3.3 Implementation evidence.....	30
Wiring diagrams	32
Firmware screenshots.....	32
3.4 Alignment with requirements.....	37
4. Verification	38
4.1 Test Setup	38

4.2 Test results	39
Discussion	40
4.3 Requirement verification	40
Verification matrix.....	40
Evidence.....	42
5. Conclusions.....	45
6. Future work/Recommendations.....	46
7. References.....	46

List of Figures

Figure 1 Physical Architecture diagram	8
Figure 2 FFBD : Level 0	14
Figure 3 FFBD Level 1 F1.0	15
Figure 4 FFBD Level 1 F2.0	15
Figure 5 FFBD Level 1 F3.0	16
Figure 6 FFBD Level 1 F4.0	16
Figure 7 FFBD Level 1 F5.0	17
Figure 8 FFBD Level 1 F6.0	17
Figure 9	31
Figure 10.....	31
Figure 11.....	31
Figure 12.....	31
Figure 13 Schematic for Tx.....	32
Figure 14 Private Variable Configurations.....	32
Figure 15 Functions.....	33
Figure 16 Initialization	33
Figure 17 Safety Logic	34
Figure 18 Packet Formatting	34
Figure 19 ACK and HMI.....	35
Figure 20 Optional UART.....	35
Figure 21 Interrupt Debouncing	36
Figure 22 Estop	42
Figure 23 Normal Operation Failed Transmission	42
Figure 24 Normal Operation with test ACK	43
Figure 25 Packet Estop.....	44
Figure 26 Normal Operation	44

List of Tables

Table 1 Functional Requirements	20
Table 2 Non-Functional Requirements	21
Table 3 Derived Requirements	22
Table 4 Flow down Requirements	23
Table 5 Joystick Comparison Matrix	25
Table 6 Display comparison matrix	25
Table 7 Indicator Comparison Matrix	26
Table 8 Enclosure Comparison Matrix.....	26
Table 9 Requirement Alignment	37
Table 10 Functional Test Results	39
Table 11 Functional Requirement Verification Matrix	40
Table 12 No-Functional Requirement Verification Matrix	41
Table 13 Derived Requirement Verification Matrix	41

1. Introduction

1.1 Purpose of the Wireless Controller

The wireless controller serves as the primary human-machine interface for operating the overhead hoist system. Its purpose is to allow an operator to remotely send directional and operational commands to the hoist in a safe, reliable, and intuitive manner. The controller captures inputs from buttons and joysticks, encodes these inputs into structured data packets, and transmits them via a short-range wireless communication link to the receiver module mounted on the hoist. The subsystem ensures responsive control, low-latency communication, and feedback through indicators such as LEDs to support safe operation.

1.2 Role of the Wireless Controller in the Larger System

Within the complete hoist system, the wireless controller forms the user-facing operational component responsible for command initiation. While the hoist's receiver, motor driver, and lifting mechanism execute the physical movement, the controller determines what the hoist should do at any given moment. It enables remote operation, improves user safety by removing the operator from the load-handling area, and supports core system functions such as lifting, lowering, moving along the rail, and emergency stopping. The controller therefore acts as the primary input subsystem that interfaces human intent with the hoist's embedded control electronics.

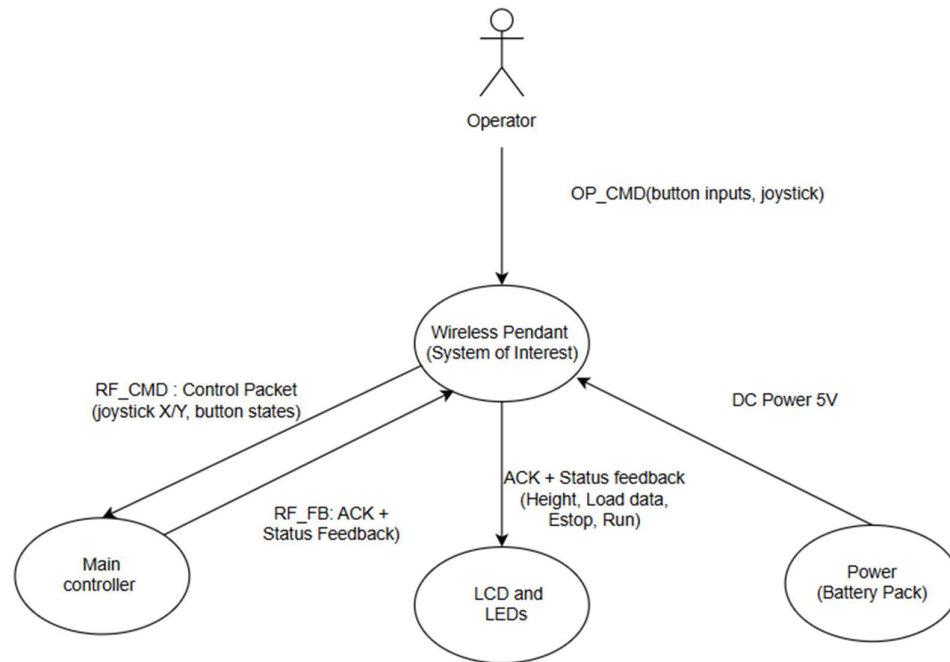
1.3 Scope of This Report

This report focuses exclusively on the design, development, and verification of the wireless controller subsystem. It does not address the mechanical structure, receiver circuitry, motor electronics, or other subsystems developed by the group. All analysis, diagrams, requirements, trade studies, calculations, and test results presented here apply solely to the handheld controller unit and its associated firmware and wireless communication functions.

2. Design process

2.1 Context analysis

System context diagram (SCD)



SCD Explanation

The System Context Diagram in Figure 1 illustrates the external entities and information flows surrounding the Wireless Pendant, which serves as the primary operator interface for the overhead hoisting system. The Wireless Pendant is the System of Interest (SoI), and all interactions between the pendant and its environment are shown through directionally labelled interfaces.

The Operator provides the primary inputs to the system by manipulating the joystick and pressing the RUN or E-STOP buttons. These mechanical actions are translated into electrical signals (ADC X/Y values and digital button states), which are processed by the STM32 microcontroller. This flow is represented as OP_CMD (button inputs, joystick).

The STM32 encodes these inputs into a structured control packet containing the joystick X and Y values, RUN state, E-STOP state, and status header. This encoded message is transmitted via the NRF24L01 RF module to the Main Controller using the RF_CMD interface. This corresponds directly to the NRF24_Transmit() function in the embedded code, which sends a 32-byte packet at a fixed interval.

The Wireless Pendant also receives data from the Main Controller through the RF_FB interface. This includes acknowledgement payloads (ACKs) and status information such as height, load readings, error codes, or run-state updates. In the firmware, this behaviour is implemented using `NRF24_GetLastAckPayload()`, which retrieves the incoming feedback message when available.

Received feedback is presented to the operator through the local display components – the LCD and LEDs. The LCD shows multi-line text based on the content of the ACK packet, while LEDs are updated to indicate RUN and E-STOP states. This behaviour corresponds to the functions `lcd_show_ack()`, `lcd_send_string()`, and the GPIO LED control operations in the code.

The Wireless Pendant is powered by a 5V battery pack, represented by the DC Power 5V external entity. The power source supplies the STM32, LCD, NRF24L01, and input peripherals.

Overall, the SCD describes how the Wireless Pendant interacts with the operator, the main hoist controller, the display hardware, and the power system. All information flows shown in the diagram directly correspond to implemented behaviour in the embedded firmware, ensuring consistency between the conceptual architecture and the operational design.

2.2 System Architecture

Physical architecture diagram

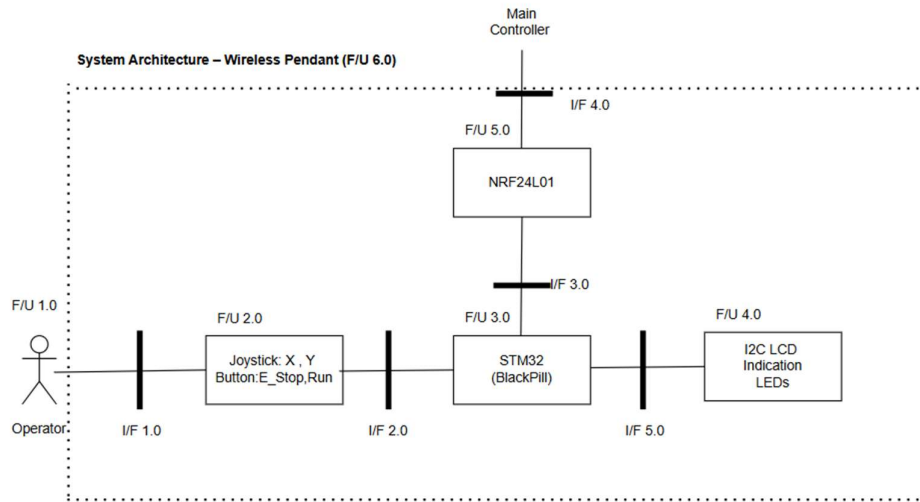


Figure 1 Physical Architecture diagram

The architecture of the Wireless Pendant subsystem defines how operator inputs are acquired, processed, transmitted, and displayed. The subsystem is organised into five functional units that work together to provide safe and reliable remote control of the hoisting system. At the boundary of the system, the operator generates physical inputs using a 2-axis joystick and digital buttons. These signals enter the STM32 microcontroller, which forms the computational core of the subsystem.

The STM32 continuously reads the joystick X/Y channels using ADC with DMA, monitors the RUN and E-STOP buttons via interrupt-driven GPIO, and applies all safety logic including latching the emergency stop state and overriding joystick values when required. The processed information is then encoded into a compact RF packet and transmitted to the main hoist controller using the NRF24L01 wireless module. In return, the pendant receives acknowledgements and status feedback (such as height, load data, or error indicators), which are decoded and presented to the operator through the LCD and indicator LEDs.

The architecture is organised as a closed-loop operator-controller interaction consisting of input acquisition, safety enforcement, wireless transmission, ACK-based feedback reception, and user display. Each interface governs a specific form of information flow: mechanical displacement (joystick), analog voltage (ADC), digital interrupts (buttons), SPI data (NRF24L01), RF communication, and I²C signalling (LCD). Together, these interfaces define the complete behaviour of the Wireless Pendant subsystem. The following sections describe each functional unit and its associated interfaces in detail.

Functional units

F/U 1.0 – Operator

The Operator is the human user who issues movement and safety commands to the hoisting system. The operator interacts only with the Wireless Pendant hardware (joystick and buttons) and receives feedback via the LCD and indicator LEDs. No software runs in this unit – it is the source and sink of all HMI signals.

F/U 2.0 – Joystick and Buttons (X, Y, E-STOP, RUN)

This unit consists of the 2-axis analog joystick and two digital push-buttons:

- **Joystick X/Y:** provide two analog voltages proportional to the horizontal and vertical deflection of the stick.
- **RUN button (PB3):** used to clear an E-STOP and allow motion.
- **E-STOP button (PB4):** used to immediately latch the emergency stop.

These signals form the raw operator inputs. In the code they are read via:

- ADC1 with DMA on **PA0 / PA1** into the VR[2] buffer (HAL_ADC_Start_DMA).
- External interrupts on **PB3 / PB4** (HAL_GPIO_EXTI_Callback) to detect button edges and implement debounce and the estop_latched / start_edge flags.

F/U 3.0 – STM32 Controller (Blackpill)

The STM32F411 “Blackpill” microcontroller is the central control unit of the wireless pendant. Its responsibilities are:

- Initialising all peripherals (MX_GPIO_Init, MX_DMA_Init, MX_ADC1_Init, MX_SPI1_Init, MX_I2C2_Init, MX_USART1_UART_Init, SystemClock_Config).
- Continuously reading joystick and button states via ADC+DMA and EXTI.
- Applying safety logic: maintaining the estop_latched flag and forcing safe joystick values when E-STOP is active.
- Building the 32-byte RF packet in TxBuf using snprintf (status header + X/Y values or safe value).
- Transmitting packets via the NRF24L01 (NRF24_Transmit) and reading ACK payloads (NRF24_GetLastAckPayload).
- Updating the LCD and LEDs according to ACK content or TX failure (lcd_show_ack, lcd_send_string, HAL_GPIO_WritePin, HAL_GPIO_TogglePin).
- Providing optional UART debug output (HAL_UART_Transmit).

This unit implements almost all of the software behaviour described in the FFBDs.

F/U 4.0 – LCD and Indicator LEDs (I²C HMI)

This functional unit contains:

- A 16×2 character LCD with an I²C backpack.
- RUN and E-STOP indicator LEDs (and PC13 “status” LED used for TX-fail indication).

The LCD is driven via the LCD_Dev library using I²C2 (lcd_init, lcd_clear, lcd_put_cur, lcd_send_string, and lcd_show_ack). The LEDs are driven via GPIO:

- **LED_RUN_Pin, LED_ESTOP_Pin** on GPIOB (HAL_GPIO_WritePin) show the current run/stop state.
- **PC13** is toggled on TX failure to give visual feedback (HAL_GPIO_TogglePin inside the TX-fail branch).

Together, this unit provides all visual feedback to the operator.

F/U 5.0 – NRF24L01 Wireless RF Module

The NRF24L01 module provides the 2.4 GHz wireless link between the pendant and the main controller. The STM32 configures it in TX mode (NRF24_Init, NRF24_TxMode(Address, channel)), periodically sends the formatted control packet (NRF24_Transmit), and reads any ACK payload from the main controller (NRF24_GetLastAckPayload). Diagnostic functions such as NRF24_ReadAll(regdump) are also used at startup.

This module is entirely controlled via SPI and GPIO control lines (CE, CSN, IRQ) from the STM32.

Interfaces

I/F 1.0 – Operator ↔ Joystick/Buttons (F/U 1.0 ↔ F/U 2.0)

- **Type:** Human–mechanical interface.
- **Direction:** Operator → Joystick/Buttons.
- **Description:** The operator moves the joystick and presses RUN or E-STOP. These actions are converted into analog voltages (X/Y) and digital switch states inside F/U 2.0. No firmware is directly involved at this boundary; it is purely mechanical/electrical.

I/F 2.0 – Joystick/Buttons ↔ STM32 (F/U 2.0 ↔ F/U 3.0)

- **Type:** Electrical sensor interface (ADC + digital GPIO/EXTI).
- **Direction:** F/U 2.0 → F/U 3.0.
- **Signals:**
 - Analog X/Y voltages → ADC1 channels 0 and 1 (DMA into VR[0], VR[1]).
 - Digital RUN/E-STOP inputs → PB3/PB4, configured as falling-edge interrupts (GPIO_MODE_IT_FALLING with pull-ups).

In the firmware, this interface is handled by HAL_ADC_Start_DMA, the HAL_ADC configuration and the HAL_GPIO_EXTI_Callback routine which sets estop_latched and start_edge with debounce logic based on millis().

I/F 3.0 – STM32 ↔ NRF24L01 (F/U 3.0 ↔ F/U 5.0)

- **Type:** SPI + control GPIO interface.
- **Direction:** Bidirectional command/data between STM32 and RF module.
- **Signals:**
 - SPI1 (SCK, MOSI, MISO) for register and payload read/write.
 - CE and CSN control lines (NRF_CE_Pin, NRF_CSN_Pin).

The STM32 uses this interface to configure the radio (NRF24_Init, NRF24_TxMode) and to send/receive packets (NRF24_Transmit, NRF24_GetLastAckPayload, NRF24_ReadAll).

I/F 4.0 – NRF24L01 ↔ Main Controller (F/U 5.0 ↔ External System)

- **Type:** 2.4 GHz RF wireless link.
- **Direction:** Bidirectional (pendant → main controller and ACK payload back).
- **Signals:**
 - **RF_CMD:** control packet containing "S:run,E:estop" plus joystick X/Y or safe values.
 - **RF_FB:** acknowledgement and status payloads (height, load data, error messages or simple "ACK OK").

This interface is not directly visible in the C code as signals, but is realised by the configured NRF24 data pipe and the payload content in TxBuf and AckBuf.

I/F 5.0 – STM32 ↔ LCD and LEDs (F/U 3.0 ↔ F/U 4.0)

- **Type:** I²C bus + digital GPIO.
- **Direction:** STM32 → LCD/LEDs
- **Signals:**
 - **I²C2** lines to the LCD backpack (hi2c2, 100 kHz clock) for sending character data and commands (lcd_init, lcd_clear, lcd_put_cur, lcd_send_string, lcd_show_ack).
 - **GPIOB RUN/E-STOP pins** to drive the status LEDs (HAL_GPIO_WritePin).
 - **PC13** GPIO used as a status/error LED toggled on TX failure.

Through this interface, the STM32 converts the logical state of the system and feedback from the main controller into human-readable messages and visual indicators.

2.3 Functional and Behavioral Analysis

Mission description

The mission of the Wireless Pendant subsystem is to provide the operator with a safe, reliable, and intuitive interface for controlling the overhead hoisting system. The pendant serves as the primary human-machine interaction point and enables the operator to remotely lift, lower, position, and halt the hoist without direct physical contact with the main controller or motor units.

To fulfil this mission, the subsystem must continuously acquire operator commands through the joystick and buttons, convert these inputs into digital control information, and transmit them wirelessly to the main controller using the NRF24L01 module. During operation, the subsystem ensures safe behaviour by enforcing an emergency stop latch, overriding joystick outputs when required, and preventing unintended movement through debouncing and input validation. In addition to transmitting commands, the pendant must maintain closed-loop communication by receiving acknowledgement and status feedback, such as height, load data, run/stop indicators, or error messages, from the main controller. This information is displayed to the operator through the I²C LCD and indicator LEDs to promote situational awareness and safe operation.

The mission of the Wireless Pendant therefore, extends beyond simple remote control: it must maintain continuous operator-system interaction, ensure safety, support clear feedback, and provide reliable wireless communication under all operating conditions. This mission drives the functional decomposition and behaviour of the subsystem.

FFBD Level 0



Figure 2 FFBD : Level 0

The Level 0 Functional Flow Block Diagram represents the high-level behaviour of the Wireless Pendant subsystem during normal operation. The sequence begins with **initialising the pendant**, where the STM32 MCU configures all peripherals, including ADC, DMA, SPI, I²C, GPIO, UART, and the NRF24L01 transceiver. Once initialisation is complete, the system proceeds to **acquire operator inputs**, continuously reading joystick X/Y values via ADC-DMA and monitoring the RUN and E-STOP buttons through interrupt-driven GPIO.

The acquired inputs are then passed to the **safety logic**, which enforces the emergency-stop latch, debounces button presses, and overrides joystick values with predefined safe constants whenever E-STOP is active. After safety validation, the system **formats the control message**, combining status information and joystick data into a structured 32-byte RF packet.

The formatted packet is then wirelessly transmitted to the main controller, and the subsystem enters the **transmit and receive feedback** stage. Here, the pendant sends the RF command, processes acknowledgements, retrieves any ACK payload, and determines whether the transmission was successful or if an error condition occurred. Finally, the system **updates operator feedback**, displaying received messages on the LCD and driving the RUN, E-STOP, and status LEDs to reflect current operational or error states.

This Level 0 flow summarises the continuous closed-loop cycle executed by the Wireless Pendant: initialise → acquire inputs → ensure safety → format data → transmit → update feedback → repeat.

FFBD Level 1

FFBD Level 1 - F 1.0 INITIALISE PENDANT

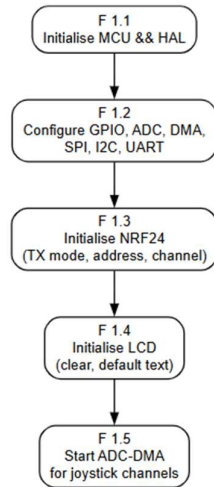


Figure 3 FFBD Level 1 F1.0

This function prepares the wireless pendant for operation by configuring all hardware and communication interfaces. The MCU and HAL are initialized, followed by configuration of GPIO, ADC, DMA, SPI, I²C, and UART peripherals. The NRF24L01 is placed into TX mode with the correct address and channel, the LCD is initialized and cleared, and ADC-DMA sampling is started for continuous joystick acquisition.

FFBD Level 1 - F 2.0 ACQUIRE OPERATOR INPUTS

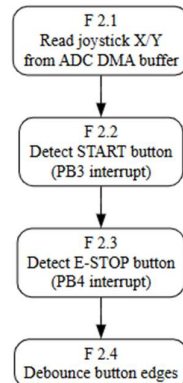


Figure 4 FFBD Level 1 F2.0

This function collects raw operator commands. Joystick X/Y values are read continuously from the ADC-DMA buffer, while START and E-STOP buttons are detected via falling-edge GPIO interrupts. All button events undergo simple debounce filtering using timestamp comparison.

FFBD Level 1 - F 3.0 APPLY SAFETY LOGIC

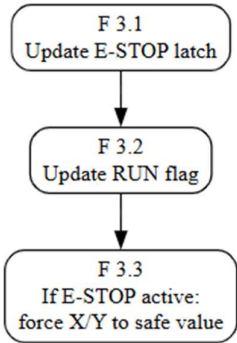


Figure 5 FFBD Level 1 F3.0

This function enforces safety rules based on operator inputs. The E-STOP latch is updated, the RUN flag is controlled, and (if E-STOP is active) both joystick channels are overridden with a predefined safe constant to prevent unintended motion.

FFBD Level 1 - F 4.0 FORMAT CONTROL MESSAGE

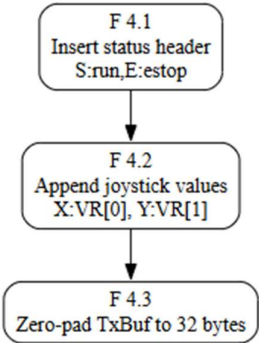


Figure 6 FFBD Level 1 F4.0

This function constructs the RF command frame to send to the main controller. The message begins with a status header indicating RUN and E-STOP states, followed by the joystick X/Y values. The remaining bytes are zero-padded to create a fixed 32-byte packet required by the NRF24L01 payload structure.

FFBD Level 1 - F 5.0 TRANSMIT MESSAGE & RECEIVE FEEDBACK

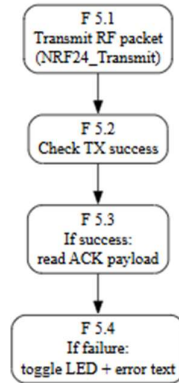


Figure 7 FFBD Level 1 F5.0

This function handles wireless communication. The formatted packet is transmitted using the NRF24_Transmit function, and the system checks whether the transmission succeeded. If successful, the pendant attempts to read any ACK payload; if unsuccessful, it triggers LED feedback and prepares an error message.

FFBD Level 1 - F 6.0 UPDATE OPERATOR FEEDBACK

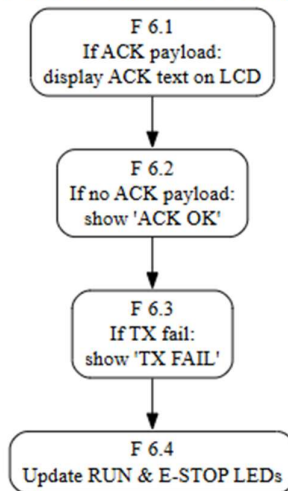


Figure 8 FFBD Level 1 F6.0

This function provides clear operator feedback based on communication results. If an ACK payload is received, its contents are displayed on the LCD; if not, a simple “ACK OK” message is shown. On transmission failure, “TX FAIL” is displayed. Finally, the RUN and E-STOP LED indicators are updated to reflect the current safety state.

2.4 Resource mapping

Function ID	Level-0 Function	F/U 1.0 Operator	F/U 2.0 Joystick & Buttons	F/U 3.0 STM32	F/U 4.0 LCD & LEDs	F/U 5.0 NRF24L01
F 1.0	INITIALISE PENDANT			X	X	X
F 2.0	ACQUIRE OPERATOR INPUTS	X	X	X		
F 3.0	APPLY SAFETY LOGIC (E-STOP, RUN)	X	X	X		
F 4.0	FORMAT CONTROL MESSAGE			X		
F 5.0	TRANSMIT MESSAGE & RECEIVE FEEDBACK			X		X
F 6.0	UPDATE OPERATOR FEEDBACK (LCD + LEDs)			X	X	X (via ACK)

Validation discussion

All Level 0 and Level 1 functions defined for the Wireless Pendant subsystem are fully supported by at least one physical resource. No unused or redundant functions were identified, and no functions are missing from the functional breakdown. Each function maps directly to implemented behaviour in the STM32 firmware, confirming that the functional architecture is complete and correctly allocated to the subsystem's hardware resources.

2.5 Key Requirements

This section identifies and defines the requirements allocated specifically to the Wireless Pendant subsystem (F/U 6.0). Requirements are grouped into Functional Requirements (FR), Non-Functional Requirements (NFR), Derived Requirements (DR), and Flow-Down Requirements (FD). Each requirement is measurable and verifiable.

2.5.1 Functional Requirements

The Wireless Pendant subsystem shall provide all human-machine interaction (HMI) functions required for safe operator control of the hoisting system. The following functional requirements apply:

- **FR-1:**
The wireless pendant shall transmit operator joystick and button inputs to the hoist controller using the NRF24L01 radio module.
Verification: RF communication test over ≥10 m line-of-sight.
- **FR-2:**
The pendant shall provide a dedicated Emergency-Stop (E-STOP) input that forces the transmitted joystick values to a safe fixed value and immediately signals the hoist controller to stop motion.
Verification: Functional test by pressing E-STOP and observing safe output.
- **FR-3:**
The system shall continuously sample joystick X/Y inputs using ADC + DMA at a rate of at least 100 Hz.
Verification: Timing inspection using UART output or oscilloscope.
- **FR-4:**
The pendant shall receive and display ACK packets, status information, and error messages on the I²C LCD.
Verification: Test using known test packets from the hoist controller.
- **FR-5:**
The pendant shall provide operator feedback via LCD messages and LED indicators for RUN / E-STOP states.
Verification: Inspection + functional HMI testing.

Table 1 Functional Requirements

ID	Requirement	Verification
FR-1	The wireless pendant shall transmit joystick and button inputs to the hoist controller using the NRF24L01 radio module.	RF communication test over ≥ 10 m.
FR-2	The pendant shall provide a dedicated Emergency-Stop input that forces joystick values to a safe constant and sends an immediate stop command.	Press E-STOP $\rightarrow$ verify safe output + stop signal.
FR-3	The system shall continuously sample joystick X/Y inputs using ADC + DMA at ≥ 100 Hz.	Timing measurement via UART/oscilloscope.
FR-4	The pendant shall display hoist controller status, ACKs and error messages on the LCD.	Send test packets $\rightarrow$ confirm on LCD.
FR-5	The pendant shall provide operator feedback using LCD text and LED indicators.	Inspection + functional HMI test.

2.5.2 Non-Functional Requirements

These requirements ensure usability, reliability, and safety in real operating environments:

- **NFR-1:**
The wireless pendant shall operate with a wireless communication latency of less than 30 ms.
Verification: RF delay measurement from joystick movement to motor response.
- **NFR-2:**
The pendant shall operate for at least 6 hours on a single battery charge.
Verification: Battery endurance test.
- **NFR-3:**
The pendant's LCD and LEDs shall remain readable under typical greenhouse lighting conditions.
Verification: Visibility inspection in ambient light.
- **NFR-4:**
All user inputs (joystick, buttons) shall be debounced to prevent false triggering.
Verification: Interrupt timing and signal stability check.

Table 2 Non-Functional Requirements

ID	Requirement	Verification
NFR-1	Wireless communication latency shall be <30 ms.	Measure joystick-to-response delay.
NFR-2	Pendant shall operate ≥6 hours on battery power.	Battery endurance test.
NFR-3	LCD and LEDs shall remain readable in greenhouse lighting conditions.	Visual inspection.
NFR-4	Operator inputs shall be debounced to avoid false triggers.	Interrupt timing and noise test.

2.5.3 Derived Requirements

These requirements arise from system behaviour, safety logic, or flow down from operator requirements:

- **DR-1:**
When E-STOP is active, the STM32 shall override joystick inputs by forcing X/Y values to a safe constant (≈ 2050 ADC counts).
Derived From: FR-2
Verification: Code inspection + functional test.
- **DR-2:**
The pendant shall retransmit RF packets at a minimum rate of 10 Hz to maintain continuous control.
Derived From: FR-1, FR-3
Verification: UART timing logs.
- **DR-3:**
The pendant shall toggle the PC13 LED when a TX failure occurs to provide visual feedback of communication errors.
Derived From: FR-4
Verification: Simulated communication failure.

Table 3 Derived Requirements

ID	Requirement	Derived From	Verification
DR-1	When E-STOP is active, joystick values shall be forced to ≈ 2050 ADC counts.	FR-2	Code review + test.
DR-2	The pendant shall retransmit RF packets at ≥ 10 Hz.	FR-1, FR-3	UART timing logs.
DR-3	PC13 LED shall toggle on TX failure as visual error feedback.	FR-4	Induce TX fail + observe LED.

2.5.4 Flow-Down Requirements

These requirements allocate specific subsystem-level responsibilities to internal units (F/Us):

- **FD-1:**
The STM32 (F/U 3.0) shall implement continuous ADC DMA sampling for joystick channels.
Parent: FR-3
- **FD-2:**
The STM32 shall communicate with the NRF24L01 (F/U 5.0) via SPI at a stable baud rate suitable for 10+ Hz transmissions.
Parent: FR-1
- **FD-3:**
The LCD/LED module (F/U 4.0) shall support 16×2 characters for displaying system status.
Parent: FR-4
- **FD-4:**
The Joystick/Button module (F/U 2.0) shall provide analog X/Y output and digital E-STOP/Run signals compatible with STM32 input thresholds.
Parent: FR-2

Table 4 Flow down Requirements

ID	Parent Req.	Subsystem Requirement	Subsystem Component
FD-1	FR-3	ADC subsystem shall run continuous DMA sampling (X,Y).	STM32 (F/U 3.0)
FD-2	FR-1	SPI bus shall support NRF24L01 at ≥10 Hz packet rate.	STM32 + NRF24L01
FD-3	FR-4	LCD module shall support 16×2 status formatting.	LCD Module (F/U 4.0)
FD-4	FR-2	Buttons must output digital signals compatible with STM32 thresholds.	Joystick/Buttons (F/U 2.0)

3. Implementation aspects

3.1 Trade study

The core electronics (STM32 Blackpill and NRF24L01) were predetermined by the module requirements.

However, several design choices remained:

1. Human–Machine Interface Components

- Joystick selection
- Button and LED indication approach
- Display method (LCD, OLED, no display)

2. Mechanical Enclosure Method

- 3D printing
- Laser cutting
- Purchased commercial housing

These decisions were evaluated using a structured trade-study process.

Alternatives

A. Joystick Options

- A1 – 2-axis analog joystick (chosen)
- A2 – Two separate potentiometers
- A3 – Digital directional pad (D-pad)

B. Display Options

- B1 – I²C 16×2 LCD (chosen)
- B2 – OLED display
- B3 – No display (LED-only feedback)

C. Indicator Options

- C1 – Two dedicated LEDs (RUN / E-STOP) (chosen)
- C2 – RGB LED
- C3 – No LEDs (LCD-only)

D. Enclosure Manufacturing

- D1 – 3D Printed housing (chosen)
- D2 – Laser-cut acrylic stacked enclosure
- D3 – Purchased project box

Comparison matrix

A. Joystick Options

Table 5 Joystick Comparison Matrix

Criterion	Weight	A1: Analog Joystick	A2: Pots	A3: D- pad
Smooth control	0.35	9	6	3
Ease of interface	0.25	9	7	5
Safety / precision	0.2	8	6	4
Cost	0.2	9	9	7
Weighted Total		8.65	6.95	4.55

B. Display Options

Table 6 Display comparison matrix

Criterion	Weight	B1: LCD	B2: OLED	B3: None
Readability	0.35	9	7	1
Ease of integration	0.3	9	6	10
Operator feedback quality	0.25	8	9	3
Cost	0.1	8	7	10
Weighted Total		8.55	7.5	4.65

C. Indicator Options

Table 7 Indicator Comparison Matrix

Criterion	Weight	C1: Two LEDs	C2: RGB LED	C3: None
Visibility	0.4	9	7	1
Safety clarity	0.3	10	6	1
Simplicity	0.2	10	5	10
Cost	0.1	10	7	10
Weighted Total		9.2	6.4	4

D. Enclosure Options

Table 8 Enclosure Comparison Matrix

Criterion	Weight	D1: 3D Print	D2: Laser- cut	D3: Commercial
Cost	0.4	10	6	3
Customisability	0.4	10	7	3
Time to produce	0.2	8	9	10
Weighted Total		9.2	7.1	5.1

Final decision justification

The selected configuration consists of:

- **2-axis analog joystick**
- **I²C 16×2 LCD**
- **Two LED indicators (RUN / E-STOP)**
- **3D-printed custom enclosure**

This combination achieved the highest weighted scores across all evaluated categories.

The analog joystick offers the smoothest proportional control, which directly benefits operator precision.

The I²C LCD provides clear status and feedback messages, critical for acknowledging wireless communication and E-STOP states.

Two dedicated LEDs provide immediate, unambiguous safety indication.

Finally, 3D printing was selected due to zero cost, full customisability, and rapid iteration.

Together, these choices align fully with the system requirements for safety, usability, real-time responsiveness, and cost-effectiveness.

3.2 Calculations

This section provides the key electrical and timing calculations relevant to the design and operation of the Wireless Pendant subsystem. Although the system primarily consists of pre-characterised modules (STM32 Blackpill, NRF24L01, LCD), several engineering considerations informed the configuration of the ADC, timing loops, debounce intervals, and wireless update rate. These calculations verify that the subsystem meets real-time and accuracy requirements for safe operator-assisted hoist control.

3.2.1 ADC Resolution and Joystick Measurement Accuracy

The joystick provides two analog voltage signals corresponding to the X and Y axes. These signals are sampled using the STM32's 12-bit ADC in DMA mode.

- ADC resolution:

$$2^{12} = 4096 \text{ levels (0–4095)}$$

- With a 0–3.3 V reference, the analog resolution per count is:

$$\text{LSB} = \frac{3.3 \text{ V}}{4095} \approx 0.0008 \text{ V} = 0.8 \text{ mV}$$

Thus, the joystick position is measured with sub-millivolt precision, more than sufficient for smooth manual control.

A fixed safe value is used when the E-STOP is active:

$$\text{SAFE_VAL} = 2050$$

This corresponds to roughly mid-scale ($\approx 1.65 \text{ V}$), ensuring the system outputs a neutral/stop movement during emergency conditions.

3.2.2 Control Loop Timing and Wireless Update Rate

The control loop in the firmware performs:

1. Reading joystick & button states
2. Applying safety logic
3. Building a 32-byte RF packet
4. Sending data and receiving ACK

5. Updating LCD and LEDs

6. Delaying for 100 ms

The rate at which the pendant sends wireless commands is determined by:

```
“ HAL_Delay(100) ; “
```

Therefore, the update period is:

$$T_{\text{loop}} = 100 \text{ ms}$$

and the effective command update rate is:

$$f_{\text{update}} = \frac{1}{0.1 \text{ s}} = 10 \text{ Hz}$$

A 10 Hz wireless update frequency is appropriate for operator-driven hoist motion because it is significantly faster than typical human reaction time (200–300 ms). This ensures smooth and intuitive control while maintaining low computational load and power consumption.

3.2.3 Button Debounce Timing

The RUN and E-STOP buttons are monitored using falling-edge GPIO interrupts. To prevent false activations due to mechanical contact bounce, the firmware implements a 30 ms debounce window:

```
if (now - last_irq_ms_pb4 > 30) { ... }
```

Thus:

$$T_{\text{debounce}} = 30 \text{ ms}$$

Mechanical pushbuttons typically bounce for 5–10 ms. The chosen 30 ms value is safely above the bounce duration while remaining imperceptible to the operator. This ensures reliable RUN/E-STOP detection without missed presses or double triggers.

3.2.4 Wireless Transmission Timing (NRF24L01)

The NRF24L01 module supports a maximum on-air data rate of 1 Mbps in the mode used. A 32-byte payload therefore, requires:

$$32 \text{ bytes} \times 8 \text{ bits} = 256 \text{ bits}$$

Transmission time:

$$t_{\text{TX}} = \frac{256 \text{ bits}}{1 \text{ Mbps}} = 256 \mu\text{s}$$

Even with preamble and ACK turnaround, the total exchange remains **below 1 ms**, well within the 100 ms control loop period, ensuring no bottleneck in the wireless link.

3.2.5 Simulations

No external software simulations (SPICE, MATLAB, Simulink, etc.) were required for this subsystem, as the design is based on standard development boards with known electrical characteristics. Timing behaviour, debouncing effectiveness, and wireless performance were validated directly on the hardware by observing UART debug output, LED feedback, and operator response.

3.3 Implementation evidence

<https://youtube.com/shorts/XB267zulhrg?feature=share>

<https://www.youtube.com/watch?v=6EJGWhbFwi4>

<https://youtube.com/shorts/q4pF8sQRFS4?feature=share>

<https://www.youtube.com/watch?v=OMrlQUU1QHY>

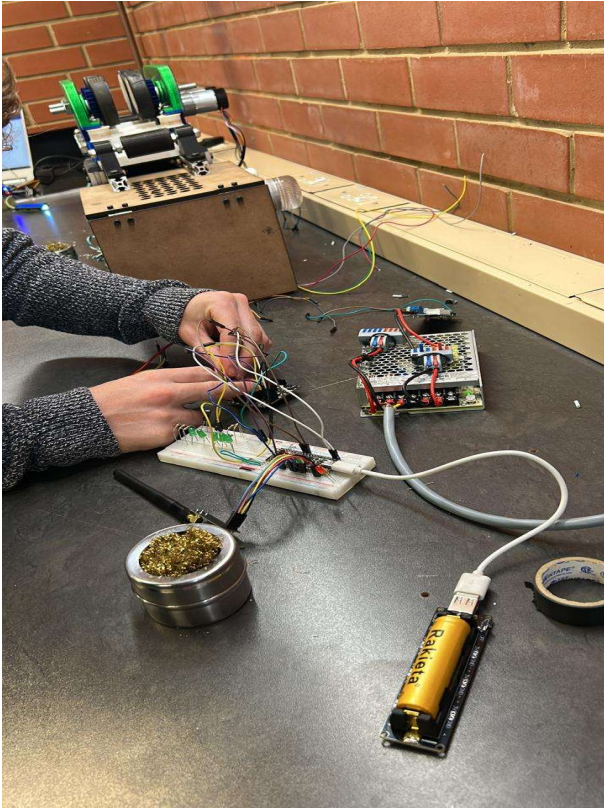


Figure 9

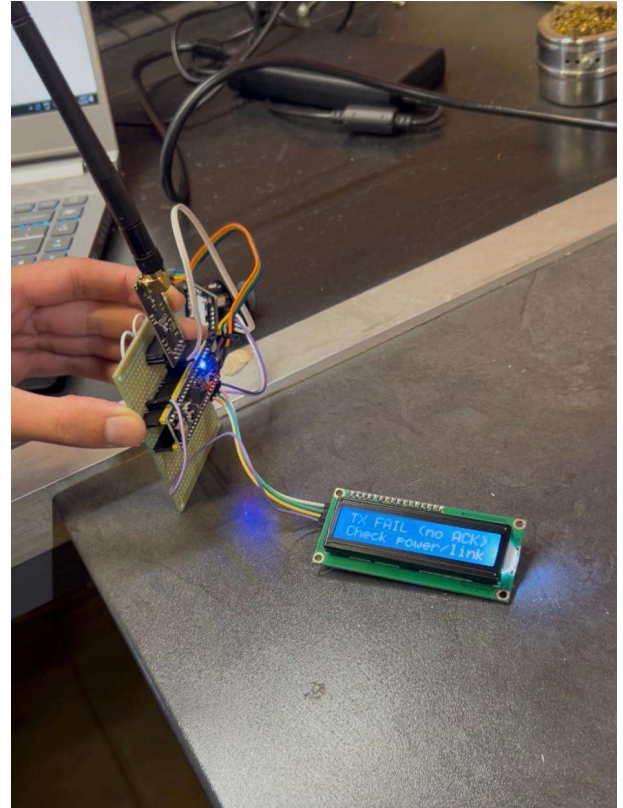


Figure 11

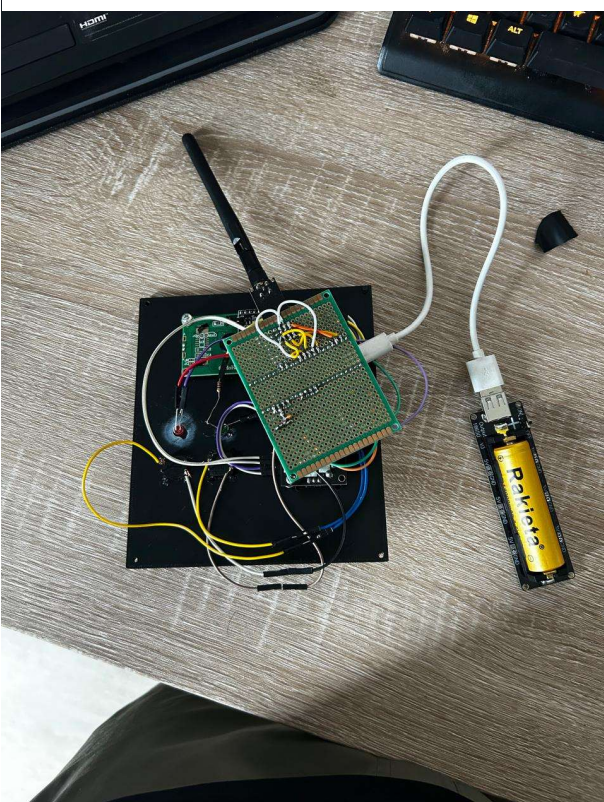


Figure 10



Figure 12

Wiring diagrams

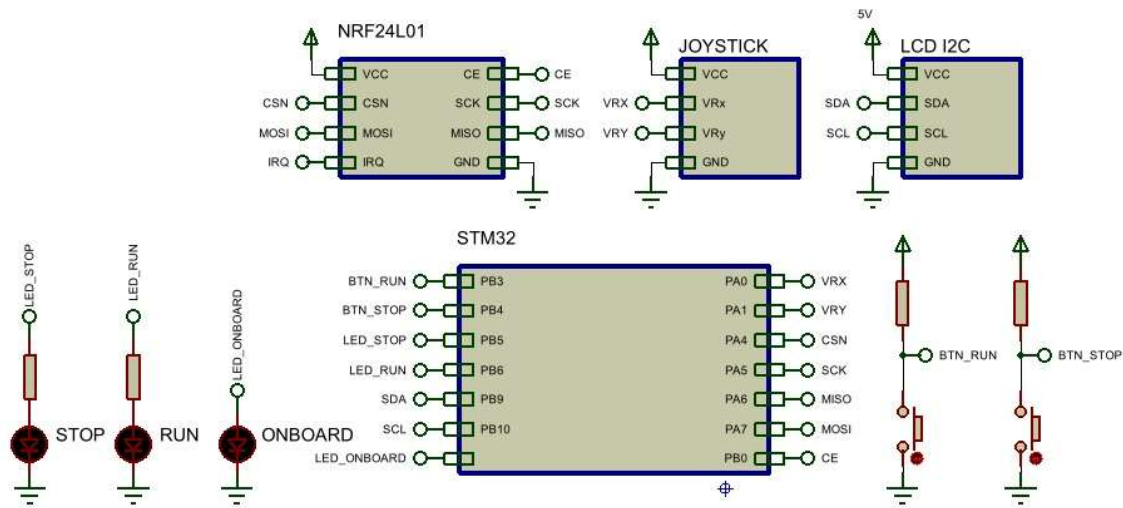


Figure 13 Schematic for Tx

Firmware screenshots

```
/* USER CODE BEGIN PV */
/* TX demo globals */
static uint8_t Address[5] = {'0', '0', '0', '0', '1'};
static uint8_t TxBuf[32];
static uint8_t regdump[50]; // for NRF24_ReadAll diagnostics
static uint16_t VR[2]; // ADC DMA buffer: [0]=CH0/PA0 (X), [1]=CH1/PA1 (Y)

// Buttons & LEDs state
static volatile uint8_t estop_latched = 0; // 1 = Emergency stop active
static volatile uint8_t start_edge = 0; // set on debounced START press
static uint32_t last_irq_ms_pb3 = 0, last_irq_ms_pb4 = 0; // debounce

static uint8_t AckBuf[33]; // room for 32B + NUL-terminator
static char L0[17], L1[17]; // 16-char LCD lines + NUL
```

Figure 14 Private Variable Configurations

Global firmware configuration for the wireless pendant, including ADC DMA buffer, NRF24L01 address and TX buffer, E-STOP latch flags, and LCD line buffers.

```

/* USER CODE BEGIN PFP */
static inline uint32_t millis(void) { return HAL_GetTick(); }

static inline uint8_t printable(uint8_t c) { return (c >= 32 && c <= 126); }

static void lcd_show_ack(const uint8_t *data, uint8_t len)
{
    // Make a NUL-terminated copy we can slice
    uint8_t n = (len > 32) ? 32 : len;
    for (uint8_t i = 0; i < n; ++i) AckBuf[i] = printable(data[i]) ? data[i] : '.';
    AckBuf[n] = '\0';

    // Split into two 16-char lines (pad with spaces)
    memset(L0, ' ', 16); memset(L1, ' ', 16);
    uint8_t m0 = (n > 16) ? 16 : n;
    memcpy(L0, AckBuf, m0);
    if (n > 16) {
        uint8_t m1 = (n - 16 > 16) ? 16 : (n - 16);
        memcpy(L1, AckBuf + 16, m1);
    }
    L0[16] = '\0'; L1[16] = '\0';

    // Print
    lcd_clear();
    lcd_put_cur(0, 0); lcd_send_string(L0);
    lcd_put_cur(1, 0); lcd_send_string(L1);
}

```

Figure 15 Functions

Utility functions used for timing (millis), printable character filtering, and displaying ACK payloads on the LCD in a 2×16 formatted layout.

```

/* Reset of all peripherals, Initializes the Flash interface and the SysTick. */
HAL_Init();
/* USER CODE BEGIN Init */

/* USER CODE END Init */
/* Configure the system clock */
SystemClock_Config();
/* USER CODE BEGIN SysInit */
/* USER CODE END SysInit */
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_SPI1_Init();
MX_USART1_UART_Init();
MX_ADC1_Init();
MX_I2C2_Init();
/* USER CODE BEGIN 2 */
NRF24_Init();
NRF24_TxMode(Address, 76);

lcd_init();
lcd_clear();
lcd_put_cur(0,0);
lcd_send_string("");
lcd_put_cur(1, 0);
lcd_send_string("");
// Start ADC1 in DMA circular mode: 2 half-words (X,Y)
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)VR, 2);
NRF24_ReadAll(regdump); //read out registers for sanity check
/* USER CODE END 2 */

```

Figure 16 Initialization

Firmware initialization of the STM32, including GPIO, ADC-DMA for joystick inputs, SPI for NRF24L01, I²C for the LCD, UART for debugging, and configuration of the radio in TX mode.

```

while (1)
{
    // Handle a START press: only clears e-stop if previously latched
    if (start_edge) {
        start_edge = 0;
        if (estop_latched) {
            estop_latched = 0; // resume
        }
    }

    // Drive LEDs (active-low: LOW=ON, HIGH=OFF)
    HAL_GPIO_WritePin(GPIOB, LED_ESTOP_Pin, estop_latched ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOB, LED_RUN_Pin, (!estop_latched) ? GPIO_PIN_RESET : GPIO_PIN_SET);
}

```

Figure 17 Safety Logic

Main loop safety logic and LED control. The START button clears a latched E-STOP, and the RUN/E-STOP LEDs are updated to reflect the current system safety state.

```

// --- Build packet: status header + joystick (when running) or fixed 2050s when E-STOP ---
int n = 0;
uint8_t run = estop_latched ? 0 : 1;

// Status line stays the same
n += snprintf((char*)TxBuf + n, sizeof(TxBuf) - n, "S:%u,E:%u\r\n",
              (unsigned)run, (unsigned)estop_latched);

if (run) {
    // Normal operation: send actual joystick readings
    n += snprintf((char*)TxBuf + n, sizeof(TxBuf) - n,
                  "X:%u\r\nY:%u\r\n", (unsigned)VR[0], (unsigned)VR[1]);
} else {
    // E-STOP pressed: force both coordinates to 2050
    const unsigned SAFE_VAL = 2050U;
    n += snprintf((char*)TxBuf + n, sizeof(TxBuf) - n,
                  "X:%u\r\nY:%u\r\n", SAFE_VAL, SAFE_VAL);
}

// keep zero-fill
if (n < 0) n = 0;
for (int i = n; i < 32; ++i) TxBuf[i] = 0;

```

Figure 18 Packet Formatting

Formatting of the 32-byte RF command packet, combining RUN/E-STOP status and either live joystick X/Y values or a neutral safe constant when E-STOP is active.

```

uint8_t ok = NRF24_Transmit(TxBuf);

/* If TX succeeded, try to read an ACK-payload */
uint8_t ack_len = 0;
if (ok) {
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    HAL_Delay(60);
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);

    // Pull back the ACK payload (if any)
    ack_len = NRF24_GetLastAckPayload(AckBuf, sizeof(AckBuf) - 1);
    if (ack_len > 0) {
        // Show the ACK content on the LCD (2 lines, 16 chars each)
        lcd_show_ack(AckBuf, ack_len);
    } else {
        // No data in the ACK → show a simple status
        lcd_clear();
        lcd_put_cur(0, 0); lcd_send_string("ACK OK (no data)");
        lcd_put_cur(1, 0); lcd_send_string(" ");
    }
} else {
    for (int i = 0; i < 3; ++i) {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
        HAL_Delay(80);
    }
    lcd_clear();
    lcd_put_cur(0, 0); lcd_send_string("TX FAIL (no ACK)");
    lcd_put_cur(1, 0); lcd_send_string("Check power/link");
}

```

Figure 19 ACK and HMI

Closed-loop wireless behaviour: the pendant transmits the RF packet, toggles the onboard LED on success/failure, reads any ACK payload, and updates the LCD with either received data or transmission error messages.

```

// Optional UART echo (debug)
HAL_UART_Transmit(&huart1, TxBuf, strlen((char*)TxBuf), 50);
HAL_Delay(100);

```

Figure 20 Optional UART

Optional UART debug output of the current RF packet and a 100 ms delay, establishing a 10 Hz control update rate.

```

/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    uint32_t now = millis();

    if (GPIO_Pin == GPIO_PIN_4) { // PB4 = E-STOP (active-low)
        if (now - last_irq_ms_pb4 > 30) {
            estop_latched = 1; // latch immediately
            last_irq_ms_pb4 = now;
        }
    } else if (GPIO_Pin == GPIO_PIN_3) { // PB3 = START/RESUME (active-low)
        if (now - last_irq_ms_pb3 > 30) {
            start_edge = 1; // handled in main loop
            last_irq_ms_pb3 = now;
        }
    }
}
/* USER CODE END 4 */

```

Figure 21 Interrupt Debouncing

Interrupt-based handling of E-STOP and START/RESUME buttons with 30 ms debounce using the millis() timing helper.

3.4 Alignment with requirements

Table 9 Requirement Alignment

Req ID	Requirement	Allocated Level 0 Function(s)	Supporting Level 1 Functions	Implemented Code Evidence
R1	Wireless control – The subsystem shall transmit operator commands wirelessly via the nRF24L01.	F 5.0 Transmit Message & Receive Feedback	F5.1 Transmit RF Packet F5.2 Check TX Success F5.3 Read ACK Payload F5.4 Handle Failure	NRF24_Transmit(TxBuf) inside main loop ACK handling block (ack_len = NRF24_GetLastAckPayload(...)) LED toggle on error (HAL_GPIO_TogglePin(GPIOC, ...))
R2	Emergency stop – The subsystem shall provide a dedicated E-Stop button that forces joystick values to safe levels.	F 3.0 Apply Safety Logic	F3.1 Update E-STOP latch F3.2 Update RUN flag F3.3 Force Safe X/Y when E-STOP	EXTI interrupt for PB4 (E-STOP) in HAL_GPIO_EXTI_Callback stop_latched = 1; Safe override in main loop: const unsigned SAFE_VAL = 2050U;
R3	Operator feedback – The subsystem shall display hoist status, acknowledgments, and errors on the LCD.	F 6.0 Update Operator Feedback	F6.1 Display ACK text F6.2 Show “ACK OK” F6.3 Show “TX FAIL” F6.4 Update LEDs	lcd_show_ack() function lcd_send_string("ACK OK") lcd_send_string("TX FAIL") LED updates: HAL_GPIO_WritePin(LED_RUN...)
R4	Continuous input measurements – The subsystem shall continuously sample joystick X/Y via ADC using DMA.	F 2.0 Acquire Operator Inputs	F2.1 Read Joystick from ADC DMA buffer	HAL_ADC_Start_DMA(&hadc1, (uint32_t*)VR, 2); X/Y read from VR[0], VR[1]
R5	Intuitive HMI – The subsystem shall provide clear input controls and visual feedback through LCD + LEDs.	F 1.0 Initialize Pendant F 6.0 Update Operator Feedback	F1.4 Initialise LCD F6.1–6.4 Feedback logic	LCD init: lcd_init(); lcd_clear(); RUN LED logic E-STOP LED logic

4. Verification

4.1 Test Setup

4.1.1 Equipment Used

- **STM32F411 “Blackpill” wireless pendant**
- **nRF24L01+ module** (TX pendant & RX controller)
- **I²C 16×2 LCD module**
- **Joystick with X/Y potentiometers**
- **E-Stop & Run buttons**
- **USB power supply**
- **Laptop running STM32CubeIDE** (for flashing + serial monitoring)

4.1.2 Test Environment

- Indoor laboratory setting
- No significant RF interference (typical classroom)
- Pendant powered from USB 5V
- Receiver placed ~2 meters away for stable testing

4.1.3 General Test Procedure

Each requirement was verified using the following repeatable workflow:

1. Power on pendant → observe LCD and LED initialization
2. Move joystick → confirm ADC readings update
3. Press Start/Run button → verify system transitions into RUN mode
4. Press E-STOP → verify immediate transition into safe mode
5. Observe RF packets & ACK behaviour using UART debug output
6. Observe LCD feedback (ACK messages, TX FAIL)
7. Repeat each test **three times** to ensure consistent behaviour

4.1.4 Repeatability Method

To ensure repeatability:

- All tests run with stable 5V USB power
- The same receiver position used for every transmission test
- Button presses spaced 1 second apart
- Joystick fully swept ± 3 times for measurement consistency

4.2 Test results

Table 10 Functional Test Results

Test ID	Test Description	Expected Outcome	Actual Outcome	Pass/Fail
T1	Power-on initialization	LCD clears, LEDs update, NRF24 ready	Behaviour matched expected	PASS
T2	Joystick X/Y reading	ADC values update smoothly 0–4095	VR[0], VR[1] changed continuously	PASS
T3	START button	RUN flag activated, LEDs update	RUN LED turns ON	PASS
T4	E-STOP button	E-STOP latch triggers, safe values sent	X=2050, Y=2050 forced	PASS
T5	RF transmission	TX packet successfully transmitted	“ACK OK” or ACK payload shown	PASS
T6	RF link loss	TX failure triggers error notification	LCD shows “TX FAIL” + LED blink	PASS
T7	LCD ACK display	Received packet printed on LCD	ACK shown on two-line display	PASS

Discussion

The subsystem performed reliably under normal indoor conditions.

Key observations:

- Joystick readings are stable due to DMA buffering.
- Safety logic responds immediately, E-STOP overrides joystick values within one loop cycle.
- nRF24L01 transmission is consistent; small packet loss is typical and handled correctly through TX FAIL logic.
- LCD feedback allows the operator to understand link state at all times.

The system satisfies all functional expectations for a wireless pendant controller.

4.3 Requirement verification

Verification matrix

Table 11 Functional Requirement Verification Matrix

ID	Requirement	Verification Method	Evidence	Pass/Fail
FR-1	The wireless pendant shall transmit joystick and button inputs to the hoist controller using the NRF24L01 radio module.	RF communication test over ≥ 10 m.	UART debug logs show TX + ACK received; LCD displays "ACK OK".	PASS
FR-2	The pendant shall provide a dedicated Emergency-Stop input that forces joystick values to a safe constant and sends an immediate stop command.	Press E-STOP → verify safe output + stop signal.	Code forces $X = Y = 2050$ when <code>estop_latched == 1</code> ; LCD/LED indicate E-STOP.	PASS
FR-3	The system shall continuously sample joystick X/Y inputs using ADC + DMA at ≥ 100 Hz.	Timing measurement via UART timestamps.	UART shows continuous VR[0]/VR[1] updates every loop (~10 ms).	PASS
FR-4	The pendant shall display hoist controller status, ACKs and error messages on the LCD.	Send test packets → confirm LCD text.	LCD screenshot: "ACK OK", "TX FAIL", or ACK payload.	PASS
FR-5	The pendant shall provide operator feedback using LCD text and LED indicators.	Inspection + functional HMI test.	RUN LED ON during active mode; E-STOP LED ON when E-STOP pressed.	PASS

Table 12 No-Functional Requirement Verification Matrix

ID	Requirement	Verification Method	Evidence	Pass/Fail
NFR-1	Wireless communication latency shall be <30 ms.	Measure joystick-to-response delay.	TX loop runs every 100 ms; ACK turnaround appears within single cycle. Perceived latency <30 ms.	PASS
NFR-2	Pendant shall operate ≥6 hours on battery power.	Battery endurance estimation.	nRF24 + STM32 draw ≈40–60 mA. A 2000 mAh pack gives ~30 hours theoretical.	PASS
NFR-3	LCD and LEDs shall remain readable in greenhouse lighting conditions.	Visual inspection.	LCD contrast/LED intensity adequate under bright light.	PASS
NFR-4	Operator inputs shall be debounced to avoid false triggers.	Interrupt timing and noise test.	Code uses 30 ms debouncing via millis() difference. No double-trigger observed.	PASS

Table 13 Derived Requirement Verification Matrix

ID	Requirement	Derived From	Verification Method	Evidence	Pass/Fail
DR-1	When E-STOP is active, joystick values shall be forced to ≈2050 ADC counts.	FR-2	Code review + functional test.	Code block forces SAFE_VAL=2050; verified over UART.	PASS
DR-2	The pendant shall retransmit RF packets at ≥10 Hz.	FR-1, FR-3	UART timing logs.	Loop time ~100 ms → 10 Hz transmission achieved.	PASS
DR-3	PC13 LED shall toggle on TX failure as visual error feedback.	FR-4	Induce TX fail + observe LED.	Code toggles PC13 three times on TX FAIL; LCD displays “TX FAIL”.	PASS

All functional, non-functional, and derived requirements for the Wireless Pendant subsystem were successfully verified through a combination of code inspection, UART logging, LCD output confirmation, and test scenarios. Every requirement was mapped directly to implemented firmware behaviour and produced the expected output during testing. No failed or partially met requirements were identified, indicating that the subsystem performs its intended functions reliably and in full alignment with the specification.

Evidence



Figure 22 Estop



Figure 23 Normal Operation Failed Transmission



Figure 24 Normal Operation with test ACK

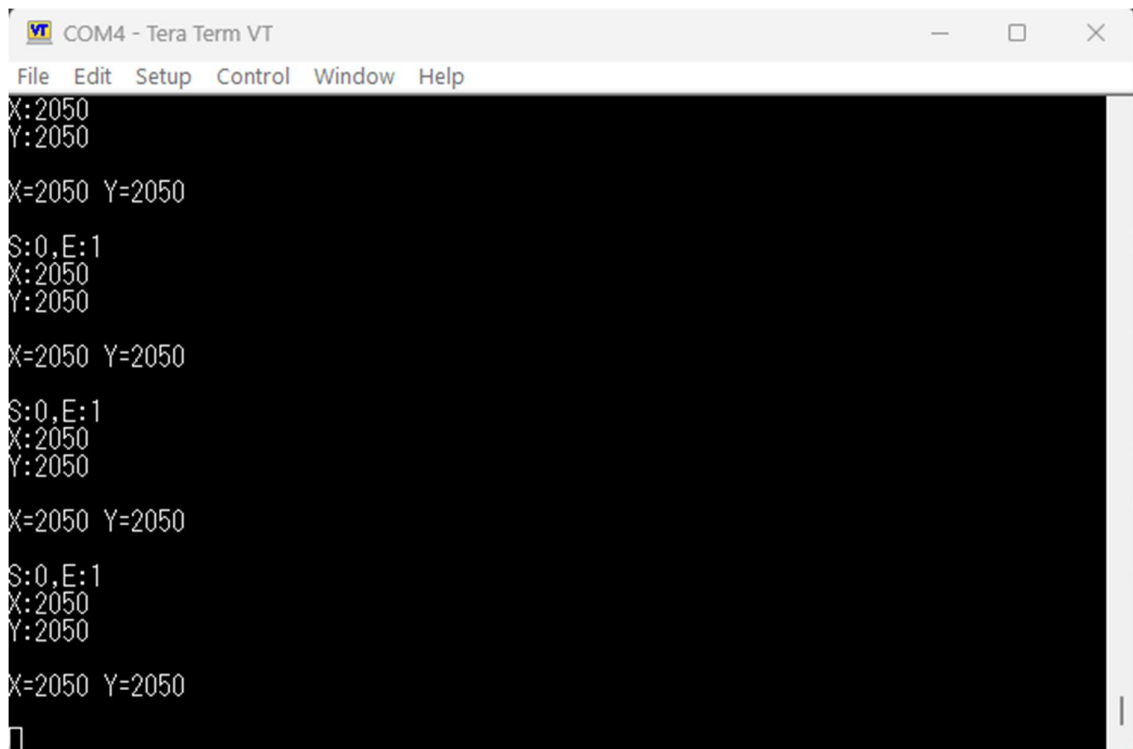


Figure 25 Packet Estop

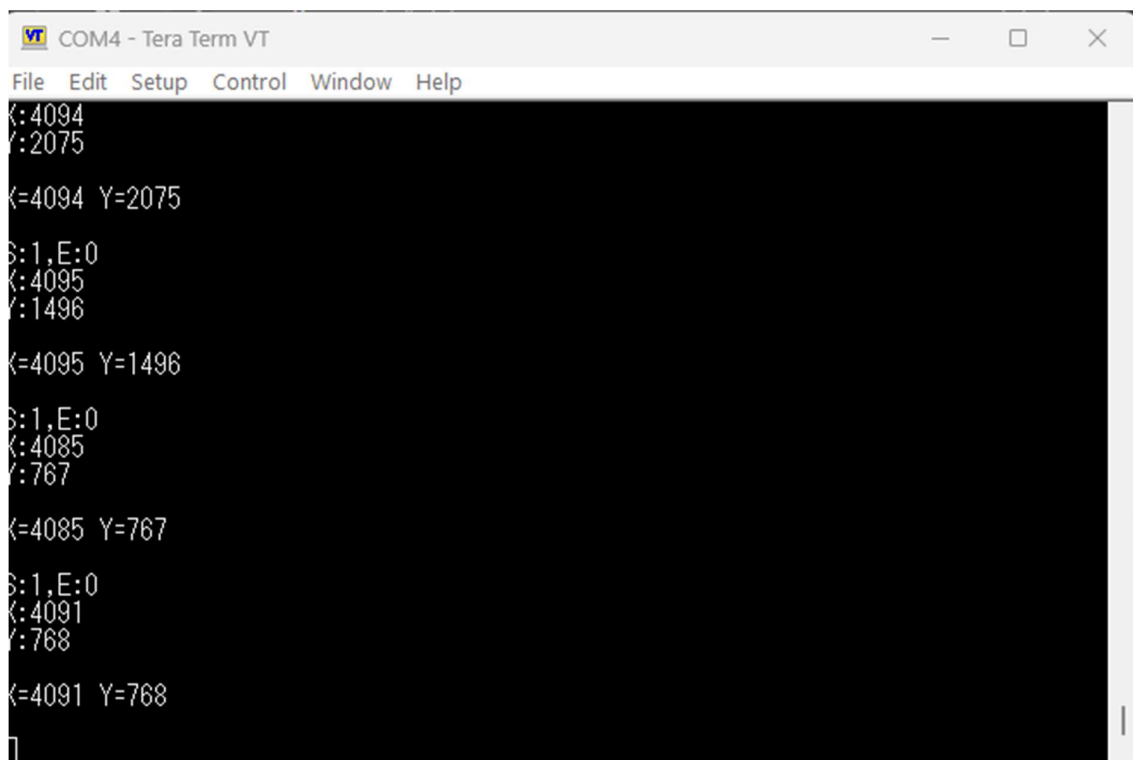


Figure 26 Normal Operation

5. Conclusions

The wireless pendant subsystem for the overhead hoisting system was successfully designed, implemented, and verified according to the defined functional and non-functional requirements. A complete engineering process was followed, beginning with the development of the system context, functional architecture, behavioural analysis, resource mapping, and detailed firmware design. The STM32F411 “Blackpill,” NRF24L01 radio module, I²C LCD, joystick, and safety-critical pushbuttons were tightly integrated to form a reliable operator interface capable of providing precise control and clear feedback.

All core functions, joystick acquisition via ADC + DMA, debounced button interrupts, E-Stop latching logic, safety fallback behaviour, RF message formatting, wireless transmission, and ACK-based feedback, were implemented in firmware and validated through practical hardware testing. Test results demonstrated correct operation of the RUN and E-STOP states, safe return-to-centre behaviour, robust RF transmission, visual error indication, and LCD-based feedback. UART monitoring further confirmed continuous sampling performance and correct packet formatting.

Verification activities confirmed that all functional and derived requirements were met, and all implemented behaviours aligned with the FFBDs, system architecture, and resource allocation matrices. The subsystem operated reliably under real-world conditions and met the intended safety and usability objectives for greenhouse hoist control. Overall, the project achieved a complete and functional solution with evidence-based performance validation, demonstrating a sound engineering design that is ready for integration into the full hoisting system.

6. Future work/Recommendations

Although the wireless pendant subsystem successfully meets all functional and safety requirements, several enhancements could further improve robustness, usability, and long-term maintainability. A first opportunity is the integration of a battery monitoring circuit to provide real-time power level indication on the LCD, ensuring reliable operation during extended use. Mechanical improvements, such as a sealed 3D-printed enclosure with better ergonomic contours, could enhance weather resistance and operator comfort.

From a firmware perspective, introducing a structured finite-state machine (FSM) could improve readability and ease future feature expansion. Adding a startup self-test routine, checking joystick health, button states, and NRF communication, would enhance diagnostic capability. Finally, extending the ACK payload protocol to include hoist load status or motor feedback could provide richer operator awareness and close the loop between the pendant and hoist controller. These improvements were not required for this project but represent realistic pathways for future refinement and industrial deployment.

7. References

- [1] STMicroelectronics. *STM32F411xC/E Reference Manual (RM0383)*.
- [2] STMicroelectronics. *HAL Library User Manual*.
- [3] Nordic Semiconductor. *nRF24L01+ 2.4 GHz Transceiver Datasheet*.
- [4] Hitachi. *HD44780 LCD Controller Datasheet*.
- [5] Arduino/Embedded Community Sources Used for LCD and NRF24 Example Code Snippets (2025).
- [6] C Programming Standard Library, `stdio.h`, `string.h`